

UNIVERSITAT POLITÈCNICA DE CATALUNYA

School of Professional & Executive Development

Postgrado en

Mobile Apps Design

PROGRAMMING WITH THREADS

Juan Luis Gorricho

PROCESSES AND THREADS

- Executing threads:
  - The initial thread or main thread is in charge of dispatching the user interface events.
  - For heavy computations it is recommended to use auxiliary threads different from the thread of the UI.
  - All events related to the UI must be executed by the **UI thread**, it's not allowed for other threads to modify the UI.
  - To interact with the UI thread we can use the **Handler** class as a communication channel among threads.

THREADS IN JAVA

- To code auxiliary threads in Java, you must create a Java class as an extension of **Thread** or implementation of the **Runnable** interface. In both cases, the Java class must overwrite the method **run**() with the intended code for the new thread.
- To start the new thread you must instantiate the Java class and invoke the method **start**(), not the method **run**(). This way we are requesting to the JVM to launch a new executing thread.

ANDROID.OS

- The **Handler** is used by the auxiliary thread sending messages thru it, as in:

```
Message msg = handler.obtainMessage();
Bundle b = new Bundle();
//put data inside the bundle
msg.setData(b);
handler.sendMessage(msg);
```

- And the UI thread will receive those messages by:

```
Handler handler = new Handler() {
    @Override
    public void handleMessage(Message msg) {
      //the processing of the message
    }
};
```

ANDROID.OS

- An alternative to sending messages with **Handler** is the method call: handler.post(Runnable), for example:

```
handler.post(doUpdateGUI);
...
// Runnable that executes the update GUI method.
private Runnable doUpdateGUI = new Runnable() {
        public void run() {
                updateGUI();
        }
};
```

ASYNCTASK

- An alternative to the coding of auxiliary threads with the Thread Java class is the use of: **AsyncTask**:

```
private class MyAsyncTask extends AsyncTask<String, Integer, Integer> {
        @Override
        protected void onProgressUpdate(Integer... progress) {
                // [... Update progress bar, Notification, or other UI element ...]
        }
        @Override
        protected void onPostExecute(Integer... result) {
                // [... Report results via UI update, Dialog, or notification ...]
        }
        @Override
        protected Integer doInBackground(String... parameter) {
                int myProgress = 0;
                // [... Perform background processing task, update myProgress ...]
                publishProgress(myProgress)
                // [... Continue performing background processing task ...]
                // Return the value to be passed to onPostExecute
                return result;
        }
}
```

TIMER, TIMERTASK

- For periodic tasks use the **Timer** class usually by calls to the scheduling methods:
  - schedule(TimerTask, delay)
  - scheduleAtFixedRate(TimerTask, delay, period)
- You also have to program a class as an extension of **TimerTask** over-writing the run() method, to specify the task to be done.
- The execution of the task is made in an independent thread.